

# Antes da Primeira Linha

*Onde o teste chega antes do código — e o projeto ganha um build de verdade*

|                 |                                                                                                                    |                                    |
|-----------------|--------------------------------------------------------------------------------------------------------------------|------------------------------------|
| NÍVEL           | Médio-difícil                                                                                                      | NOVA<br>FERRAMENTA +<br>NOVA ORDEM |
| STACK           | Java 21 + Maven 3.9 + JUnit 6                                                                                      |                                    |
| SOLUÇÃO EM      | java/04-colecoes-streams/acervo-da-quarentena/ (reestruturado)                                                     |                                    |
| TEMPO ESTIMADO  | 5 a 8 horas, em três sentadas (um Ato por sentada)                                                                 |                                    |
| PRÉ-REQUISITO   | desafio 02 aprovado — a suíte que você escreveu é o alicerce deste                                                 |                                    |
| CONCEITOS NOVOS | pom.xml, mvn test, ciclo red-green-refactor, @ParameterizedTest, @BeforeEach, injeção de Clock, teste de fronteira |                                    |

## 1. O contexto

O acervo está catalogado e tem rede. Na semana passada a Maria conferiu os números e não achou nada — o que, vindo dela, é elogio. Agora apareceu o problema seguinte, e ele não é técnico: as pessoas começaram a **levar as coisas para casa**. Uma partitura sumiu por três semanas. O DVD do Curtis e Viper 2 voltou riscado. Ninguém sabe quem está com o quê.

A comunidade quer um balcão de empréstimo. Regras simples, ditas em voz alta na reunião: item ilegível não sai; o que já saiu não sai de novo; e no fim do mês alguém precisa saber quais itens estão atrasados.

*"E se der problema, a gente conserta depois." — "Não. Desta vez a gente escreve o problema antes."*

Nos dois desafios anteriores você fez na ordem natural: construiu, depois testou. Funciona, e foi assim que você aprendeu JUnit. Mas repare no que aconteceu no desafio 02 — você escreveu um teste que não podia falhar e só descobriu porque alguém foi mutar o seu código. Isso não é azar: é consequência de escrever o teste *depois*, quando você já sabe que o código funciona e a sua cabeça está procurando confirmação em vez de contradição.

Neste desafio a ordem inverte. O teste vem primeiro, e ele precisa **falhar** antes de você escrever a implementação. Um teste que nunca foi visto vermelho é um teste em que ninguém deveria confiar — e agora você vai ter a prova de que o seu foi.

## 2. O que você vai construir

Três atos, ordem obrigatória. O Ato I não muda comportamento nenhum; o Ato II não adiciona funcionalidade nenhuma; o Ato III é onde nasce código novo — e nasce depois do teste.

| ATO                      | O QUE É                                                                                                                                                                 | POR QUE NESTA ORDEM                                                                                                               |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <b>I — A bancada</b>     | Migrar o projeto para Maven. <code>pom.xml</code> , <code>src/main/java</code> , <code>src/test/java</code> , <code>mvn test</code> verde com os seus 30 testes atuais. | Zero mudança de lógica. Se algum teste quebrar, foi a migração — e você quer descobrir isso agora, não misturado com código novo. |
| <b>II — Afiar a rede</b> | Consertar as fraquezas que a sua suíte ainda tem: fronteira, <code>@BeforeEach</code> , testes parametrizados.                                                          | Você vai apoiar código novo nesta rede. Ela precisa estar afiada antes, não depois.                                               |
| <b>III — O balcão</b>    | A funcionalidade de empréstimo, escrita <b>teste primeiro</b> , em ciclos <i>red-green-refactor</i> com prova no Git.                                                   | É o desafio. Os dois primeiros atos existem para que este seja possível.                                                          |

### Sobre o seu histórico de endereços

No desafio 02, três entregas suas estavam *certas no conteúdo e erradas no lugar*: `mutation.md` em vez de `MUTACOES.md`, testes em `test/src/` quando o desenho era `test/`, e um `ESTRUTURA.md` novo dentro da pasta do desafio enquanto o da raiz seguia desatualizado. Não é detalhe: em time, arquivo no lugar errado é arquivo que ninguém acha, e documentação duplicada é documentação que diverge.

Por isso, neste enunciado, **todo caminho está escrito por extenso e a partir da raiz do repositório**. Confira cada um antes de dizer "pronto".

## 3. Ato I — A bancada (Maven)

Até aqui você compilou com um comando gigante e um `.jar` baixado à mão. Funcionou para aprender, e agora atrapalha: não dá para declarar dependências, não dá para versionar o build, não dá para outra pessoa rodar o seu projeto sem receber instruções por mensagem.

### 3.1 A nova estrutura

```

java/04-colecoes-streams/acervo-da-quarentena/
├── pom.xml                # NOVO – o build
├── PLAN.md
├── src/
│   ├── main/java/        # o que era src/
│   │   ├── Main.java
│   │   ├── abstracts/Item.java
│   │   ├── entities/...
│   │   ├── exceptions/...
│   │   └── utils/...
│   └── test/java/        # o que era test/src/
│       ├── ItemTest.java
│       ├── MusicalScoreTest.java
│       └── ...
├── MUTACOES.md           # sobe de test/ para a raiz do projeto
└── TDD.md                # NOVO – o diário do Ato III

```

Repare que `src/main/java` e `src/test/java` não são invenção sua nem minha: é a *convenção sobre configuração* do Maven. Você não precisa dizer ao Maven onde estão os fontes; ele já sabe, porque todo projeto Java do mundo usa

esse mesmo desenho. É a mesma estrutura que você vai encontrar em qualquer projeto Spring.

### 3.2 O `pom.xml`

Não vou te dar o arquivo pronto. Vou te dar a lista do que ele precisa conter, e você escreve:

- `groupId`, `artifactId`, `version` — a identidade do projeto.
- `maven.compiler.release` = 21, e `project.build.sourceEncoding` = UTF-8. Sem o segundo, os seus acentos viram lixo em outra máquina.
- A dependência do JUnit, em escopo `test`. Procure por `org.junit:junit-bom` no Maven Central e use-o em `<dependencyManagement>` — na data em que escrevo, a versão é **6.1.3**, a mesma do `.jar` que você baixou no desafio 02. **Confira se saiu mais nova.**
- `maven-surefire-plugin` (o que roda os testes) e `maven-compiler-plugin`. Estáveis hoje: **3.5.6** e **3.16.0**. Confira também.
- Configure o compilador para emitir `-Xlint:all`. Você limpou os quatro avisos no desafio 02 — agora faça o build lembrar disso por você.

#### Uma pergunta para você responder no `PLAN.md`

O que `<scope>test</scope>` muda, na prática? O que aconteceria se você esquecesse essa linha? Responda antes de pesquisar a resposta bonita: pense no que vai parar dentro do `.jar` que você entrega para o cliente.

### 3.3 Como saber que o Ato I terminou

```
cd java/04-colecoes-streams/acervo-da-quarentena
mvn -q test
```

Precisa terminar com `Tests run: 30, Failures: 0, Errors: 0, Skipped: 0` e `BUILD SUCCESS`. Nenhuma linha de lógica alterada — o `git diff` do Ato I deve mostrar arquivos *movidos*, o `pom.xml` *criado*, e nada mais. Use `git mv`: assim o Git registra rename em vez de "apagou 9 arquivos e criou 9".

Ajuste o `.gitignore`: sai `junit-console.jar` (apague o arquivo, o Maven baixa sozinho agora), entra `target/`.

## 4. Ato II — Afiar a rede

A sua suíte do desafio 02 está boa e tem três buracos que eu encontrei mutando o seu código. Nenhum deles é difícil de fechar. Todos ensinam algo diferente.

### 4.1 O buraco da fronteira

A sua validação de ano aceita `1500..2013`. Os seus testes usam **1200** e **2049**. Eu troquei `year > 2013` por `year > 2014` no seu código e **a sua suíte inteira continuou verde**.

Pense em onde os bugs realmente moram: quase nunca no meio do intervalo, quase sempre na borda — o `<` que devia ser `<=`, o `-1` esquecido. Testar 1200 prova pouco; testar o par que cerca a fronteira prova o comportamento.

Feche esse buraco no ano, e depois olhe com o mesmo olho para `diff` (faixa 1–5) e `bpm`. Quando terminar, refaça a mutação `2013 → 2014` e confirme que agora alguém fica vermelho.

## 4.2 `@BeforeEach`, e por que não `@BeforeAll`

Três das suas classes de teste usam `@BeforeAll` com um campo `static` compartilhado. Rodei a sua suíte com ordem aleatória de método e de classe: passou nas três vezes. Ou seja, **hoje não quebra**.

Então por que mudar? Porque "hoje não quebra" e "não pode quebrar" são frases diferentes, e a distância entre elas é um teste novo que alguém escreve daqui a seis meses. Faça este exercício antes de trocar: acrescente à `CollectionStoreTest` um teste que **catalogue** um item novo no acervo compartilhado. Rode a suíte algumas vezes com ordem aleatória. Depois troque para `@BeforeEach` e rode de novo.

Escreva no `PLAN.md` o que você viu, em duas ou três linhas. É o tipo de coisa que só convence quando acontece na sua tela.

## 4.3 Testes parametrizados

Você tem oito testes de construtor em `MusicalScoreTest` que diferem só nos números e no que esperam. Isso é candidato natural a `@ParameterizedTest` com `@CsvSource`.

Converta. E então **compare as duas versões e tome partido**: ficou mais legível ou menos? Quando um caso falha, qual das duas te diz mais rápido qual caso foi? Existe resposta certa aqui, mas ela depende do caso — e eu quero a sua, escrita no `PLAN.md`, não a do primeiro blog que você abrir.

## 4.4 A dívida mais antiga do projeto

Critério de aceite 13 do desafio 01, aberto desde a primeira entrega: **um idioma só**. Hoje convivem `duracao`, `sessao`, `valids`, `availibleMinutes`, o campo `itens`, mensagens como `"duração needs greater than zero"`, e os typos `greatter` e `DEMAGED`. Escolha um idioma e aplique no projeto inteiro — código, testes e mensagens.

Agora você tem `mvn test` e uma suíte afiada. Renomeie uma coisa de cada vez, rode, siga. É para ser entediante. Entediante é o objetivo — e é a diferença entre esta rodada e a 1.5, quando você mexeu no código sem rede e introduziu duas regressões seguidas sem perceber.

## 5. Ato III — O balcão, teste primeiro

---

Aqui nasce código novo. E a regra é uma só:

## A regra do Ato III

**Nenhuma linha de implementação antes de existir um teste vermelho pedindo por ela.**

O ciclo tem três tempos, sempre nesta ordem:

- **Red** — escreva o teste. Rode. Ele precisa *falhar*. Se passou de primeira, o teste está errado ou o comportamento já existia.
- **Green** — escreva o *mínimo* que faz passar. Feio pode. Ineficiente pode. Errado para outros casos pode — esses casos ainda não têm teste.
- **Refactor** — agora arrume, com a suíte verde te avisando.

## 5.1 O que o balcão faz

Uma classe nova — o nome é seu — que cuida de empréstimos sobre o acervo que já existe. Comportamentos:

| # | COMPORTAMENTO                             | DETALHE QUE IMPORTA                            |
|---|-------------------------------------------|------------------------------------------------|
| 1 | Emprestar um item a uma pessoa            | guarda quem levou, quando saiu e o prazo       |
| 2 | Item ilegível nunca sai                   | a mesma regra da sessão da noite, agora aqui   |
| 3 | Item já emprestado não sai de novo        | sua exceção, com mensagem útil                 |
| 4 | Item que não existe no acervo não sai     | reaproveite o que o acervo já sabe fazer       |
| 5 | Devolver                                  | o item volta a ficar disponível                |
| 6 | Devolver o que não está emprestado é erro | decida qual, e defenda                         |
| 7 | Consultar se um item está disponível      | —                                              |
| 8 | Listar o que está emprestado              | cópia defensiva, como você já sabe fazer       |
| 9 | <b>Listar os empréstimos atrasados</b>    | leia a seção 5.2 <i>antes</i> de escrever este |

## 5.2 O problema do "hoje"

O comportamento 9 pergunta quais empréstimos passaram do prazo. Para responder, o seu código precisa saber que dia é hoje. A forma óbvia é chamar `LocalDate.now()` lá dentro.

Tente escrever o teste desse jeito. Sério, tente — e repare no que acontece: para testar um empréstimo atrasado você precisa que "hoje" seja depois do prazo, e você não controla o relógio da máquina. As saídas ruins aparecem sozinhas: mudar o relógio do computador, esperar de verdade, ou emprestar com data no passado e torcer.

### A pergunta, e ela vale a sessão inteira

Se o código pergunta as horas para o mundo, o teste não tem como responder. E se o *mundo* fosse um parâmetro?

Procure `java.time.Clock` — em especial `Clock.fixed(...)` e `Clock.systemDefaultZone()`. Depois pense em quem deveria criar esse `Clock`: a sua classe, ou quem a constrói?

Isto tem nome — *injeção de dependência* — e é a mesma ideia que sustenta o Spring inteiro, que é para onde você vai depois. A versão curta: um objeto não busca o que precisa, ele recebe. Quando recebe, dá para entregar outra coisa no teste.

### 5.3 A prova de que você fez teste primeiro

No desafio 02 eu conferi a ordem dos Atos pelo histórico do Git, e ela conferiu. Vamos usar o mesmo mecanismo aqui, agora de propósito. Cada ciclo do Ato III são **dois commits**:

```
test: balcão recusa emprestar item ilegível    ← suíte VERMELHA neste commit
feat: balcão recusa emprestar item ilegível    ← suíte VERDE neste commit
```

Eu vou fazer `git checkout` em alguns dos seus commits `test:` e rodar `mvn test`. Eles precisam falhar. Um commit `test:` que já passa significa que o código veio antes — e aí o Ato III não aconteceu, só foi narrado.

Mantenha também um `TDD.md` curto na raiz da pasta do desafio, uma linha por ciclo: o que o teste pedia, e o que você escreveu de mínimo para ele passar. É onde você vai perceber, relendo, quantas vezes escreveu mais do que precisava.

## 6. Critérios de aceite

- ☐ `mvn test` na pasta do desafio termina em `BUILD SUCCESS`, sem falhas e sem testes pulados.
- ☐ Estrutura Maven: `src/main/java` e `src/test/java`. O `junit-console.jar` foi apagado; `target/` está no `.gitignore`.
- ☐ O build compila com `-Xlint:all` e **zero avisos**.
- ☐ A mutação `year > 2013` → `year > 2014` deixa a suíte **vermelha**. O mesmo para as fronteiras de `diff` e `bpm`.
- ☐ Nenhuma classe de teste usa `@BeforeAll` com estado compartilhado mutável.
- ☐ Os testes de construtor de `MusicalScore` estão parametrizados.
- ☐ Idioma consistente em todo o projeto — código, testes e mensagens de exceção. Os tipos `greater` e `DEMAGED` não existem mais. (*Critério 13 do desafio 01, finalmente.*)
- ☐ Os 9 comportamentos do balcão têm teste, e os testes vieram antes — provado pelos pares de commits `test:` / `feat:`.
- ☐ Existe pelo menos um commit `test:` que, em `git checkout`, faz `mvn test` falhar.
- ☐ Os empréstimos atrasados são testados sem depender do relógio da máquina.
- ☐ `java/04-colecoes-streams/acervo-da-quarentena/TDD.md` existe, com uma linha por ciclo.
- ☐ `PLAN.md` responde: o que `<scope>test</scope>` muda; o que você viu no experimento do `@BeforeAll`; e se o parametrizado ficou melhor ou pior que os oito testes separados.
- ☐ **`docs/ESTRUTURA.md`, na raiz do repositório**, descreve a nova estrutura Maven. Se você preferir manter também o `ESTRUTURA.md` local do desafio, tudo bem — desde que os dois não se contradigam.
- ☐ `java/04-colecoes-streams/.serena/` e qualquer outro resíduo de ferramenta estão ignorados ou removidos. Nada de artefato de build no commit.

### O que não vale

- Escrever a implementação do balcão e depois o teste, com os commits arrumados para parecer TDD. Eu rodo os commits.
- Fazer o Ato III antes do Ato I ou do Ato II.
- Apagar, comentar ou afrouxar teste para ele passar. Você já enfrentou essa porta duas vezes e escolheu certo nas duas.
- Copiar um `pom.xml` de blog sem saber explicar cada bloco. No review, eu pergunto o que cada um faz.
- Pedir a implementação pronta ao Claude. Erro de build, "o que é `dependencyManagement`", "por que meu teste não é encontrado": pode e deve perguntar.

## 7. Bônus — depois que o principal estiver de pé

---

- Faça o `mvn test` **falhar o build** quando houver aviso do compilador. Dica: `-Werror`. É assim que pipeline sério funciona.
- Use `@Nested` para agrupar os testes do balcão por cenário (empréstimo, devolução, atraso).
- Adicione o plugin do **PIT** (`pitest-maven`) e rode `mvn pitest:mutationCoverage`. É a ferramenta que faz automaticamente o que eu venho fazendo à mão nos seus dois desafios. Compare o número dela com a sua intuição.
- Transforme `Item` numa `sealed class`. O que o compilador passa a garantir? O que você perde?

## 8. Por que este desafio existe

---

Duas coisas, e elas são independentes.

A primeira é o Maven, e é a menor das duas. É configuração, você aprende em uma tarde, e a partir dela todo projeto Java do mundo passa a ter a mesma cara para você — inclusive o Spring, que é o próximo destino. Não é intelectualmente interessante; é infraestrutura, como tomada de parede.

A segunda é a que importa. **Escrever o teste antes muda o código que você escreve**, e não dá para acreditar nisso por argumento — só sentindo. Quando o teste vem primeiro, você é obrigado a decidir como a sua classe vai ser *usada* antes de decidir como ela vai ser *feita*, e classes difíceis de testar viram óbvias cedo, quando ainda custa barato mudar. O comportamento 9 deste desafio foi desenhado exatamente para você bater nessa parede: você vai querer chamar `LocalDate.now()`, vai descobrir que não consegue testar, e vai inverter a dependência. Se o teste tivesse vindo depois, você teria escrito `LocalDate.now()`, testado o que fosse possível, e seguido a vida com um pedaço não testável no meio.

No mercado isso é conversa de entrevista sênior, e cai quase sempre na mesma forma: "como você testaria uma regra que depende da data de hoje?". Quem responde "mudo o relógio da máquina" é júnior. Quem responde "injeta um `Clock`" acabou de dizer que entende inversão de dependência, e a conversa segue para Spring naturalmente. Você vai poder responder a segunda — e, melhor, vai ter batido a cabeça para chegar nela.

*Código difícil de testar quase nunca é código com um problema de teste. É código com um problema de desenho, avisando cedo.*

---

*Mestre Yoda · Desafio 03 · Primeiro a pergunta. A resposta vem quando souber o que perguntou.*